

```

from __future__ import annotations

import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.metrics import precision_recall_fscore_support, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

PROJECT_ROOT = Path(__file__).resolve().parents[1]
INPUT_PATH = PROJECT_ROOT / "data" / "processed_access_logs.csv"
SCORED_PATH = PROJECT_ROOT / "data" / "scored_access_logs.csv"
MODEL_PATH = PROJECT_ROOT / "models" / "anomaly_detection_models.joblib"
METRICS_PATH = PROJECT_ROOT / "reports" / "detection_metrics.json"

RANDOM_STATE = 42

def temporal_train_test_split(data: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Split by time, keeping future events completely out of model fitting."""
    ordered = data.sort_values("timestamp").reset_index(drop=True)
    split_index = int(len(ordered) * 0.75)
    train_data = ordered.iloc[:split_index].copy()
    test_data = ordered.iloc[split_index:].copy()

    if train_data["is_anomaly"].nunique() < 2 or test_data["is_anomaly"].nunique() < 2:
        raise ValueError("Temporal split must contain normal and anomalous events.")
    return train_data, test_data

# Inputs to the unsupervised baseline-profiling model.
ISOLATION_FEATURES = [
    "hour_deviation",
    "bytes_zscore",
    "new_location",
    "new_resource",
    "new_device",
    "new_source_ip",
    "new_command_sequence",
    "source_ip_attempts_1h",
    "source_ip_unique_entities_1h",
    "entity_events_24h",
    "is_off_hours",
    "failed_auth",
    "impossible_travel_flag",
    "travel_speed_kmh",
]

# Inputs to supervised detection and attack-type classification.
NUMERIC_FEATURES = [
    "hour",
    "is_weekend",
    "is_off_hours",
    "failed_auth",
    "session_duration",
    "bytes_transferred",
    "hour_deviation",
    "bytes_zscore",
    "new_location",
    "new_resource",
    "new_device",
    "new_source_ip",
    "new_command_sequence",
    "source_ip_attempts_1h",
    "source_ip_unique_entities_1h",
    "entity_events_24h",
    "cold_start_entity",
    "distance_from_previous_kmh",
    "travel_speed_kmh",
    "impossible_travel_flag",
]

CATEGORICAL_FEATURES = [
    "entity_type",
    "auth_method",
    "auth_result",
    "protocol",
]

MODEL_FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

def make_preprocessor() -> ColumnTransformer:
    """Apply standardisation to numbers and one-hot encoding to categories."""
    return ColumnTransformer(
        transformers=[
            ("numeric", StandardScaler(), NUMERIC_FEATURES),
            (
                "categorical",
                OneHotEncoder(handle_unknown="ignore"),
                CATEGORICAL_FEATURES,
            ),
        ]
    )

def make_random_forest() -> Pipeline:
    """Create a balanced random-forest classifier."""
    return Pipeline(
        steps=[
            ("preprocessor", make_preprocessor()),
            (
                "classifier",
                RandomForestClassifier(
                    n_estimators=350,
                    max_depth=16,
                    min_samples_leaf=2,
                    class_weight="balanced_subsample",
                    random_state=RANDOM_STATE,
                    n_jobs=-1,
                ),
            ),
        ]
    )

def explain_event(row: pd.Series) -> str:
    """

```

```

Produce analyst-readable reasons for an alert.
These rule-based factors complement the ML probability.
"""
factors = []

if row["impossible_travel_flag"] == 1:
    factors.append(
        f"impossible travel ({row['distance_from_previous_km']:.0f} km)"
    )

if row["failed_auth"] == 1 and row["source_ip_attempts_1h"] >= 6:
    factors.append(
        f"{int(row['source_ip_attempts_1h'])} failed attempts from one IP/hour"
    )

if row["source_ip_unique_entities_1h"] >= 8:
    factors.append(
        f"one IP targeted {int(row['source_ip_unique_entities_1h'])} entities"
    )

if row["new_device"] == 1:
    factors.append("new device fingerprint")

if row["new_location"] == 1:
    factors.append("new geographic location")

if row["new_source_ip"] == 1:
    factors.append("new source IP")

if row["new_resource"] == 1:
    factors.append("resource outside normal entity profile")

if row["new_command_sequence"] == 1:
    factors.append("unusual command sequence")

if row["is_off_hours"] == 1:
    factors.append("off-hours activity")

if row["bytes_zscore"] >= 3:
    factors.append("unusually large data transfer")

if not factors:
    return "No strong rule-based deviation; alert derives from behaviour model."

return "; ".join(factors[:4])

def risk_tier(score: float) -> str:
    if score >= 75:
        return "Critical"
    if score >= 50:
        return "High"
    if score >= 25:
        return "Medium"
    return "Low"

def predict_attack_types(data: pd.DataFrame, model: Pipeline) -> np.ndarray:
    """Classify attack families, using high-confidence sequence signatures first.

    The classifier remains the fallback for mixed behaviour. These signatures
    preserve the analyst-meaningful distinction between attack families when a
    temporal pattern is unambiguous (for example, one IP targeting many users).
    """
    predictions = np.asarray(model.predict(data[MODEL_FEATURES]), dtype=object)

    credential_stuffing = (
        (data["failed_auth"] == 1)
        & (data["source_ip_unique_entities_1h"] >= 8)
    )
    brute_force = (
        (data["failed_auth"] == 1)
        & (data["source_ip_attempts_1h"] >= 6)
        & ~credential_stuffing
    )
    impossible_travel = data["impossible_travel_flag"] == 1
    device_spoofing = data["new_device"] == 1
    low_and_slow = (
        (data["is_off_hours"] == 1)
        & (data["bytes_zscore"] >= 3)
        & (data["new_command_sequence"] == 1)
    )
    lateral_movement = (
        (data["new_resource"] == 1)
        & (data["new_command_sequence"] == 1)
        & (data["new_source_ip"] == 1)
        & (data["bytes_zscore"] >= 2)
        & ~low_and_slow
    )

    predictions[credential_stuffing] = "credential_stuffing"
    predictions[brute_force] = "brute_force"
    predictions[impossible_travel] = "impossible_travel"
    predictions[device_spoofing] = "device_spoofing"
    predictions[low_and_slow] = "low_and_slow_exfiltration"
    predictions[lateral_movement] = "lateral_movement"
    return predictions

def apply_cold_start_guard(
    data: pd.DataFrame, risk_scores: np.ndarray
) -> np.ndarray:
    """Avoid alerting a new identity for normal first-use behaviour alone.

    Population-profile deviations still contribute to the score. The guard is
    applied only when there is no strong security signal, so a cold-start event
    with failed login attempts, travel anomalies, an unmanaged device, a burst,
    or an unusually large transfer continues to alert normally.
    """
    safe_context = (
        (data["cold_start_entity"] == 1)
        & (data["failed_auth"] == 0)
        & (data["impossible_travel_flag"] == 0)
        & (data["new_device"] == 0)
        & (data["source_ip_attempts_1h"] < 3)
        & (data["bytes_zscore"] < 3)
        & (data["is_off_hours"] == 0)
    )
    data["cold_start_safe_context"] = safe_context.astype(int)
    return np.where(safe_context, np.minimum(risk_scores, 20), risk_scores)

def main() -> None:

```

```

if not INPUT_PATH.exists():
    raise FileNotFoundError(
        "Processed data is missing. Run: python src\\features.py"
    )

data = pd.read_csv(INPUT_PATH, parse_dates=["timestamp"])
data = data.sort_values("timestamp").reset_index(drop=True)

# Future events are kept aside for honest sequential-data evaluation.
train_data, test_data = temporal_train_test_split(data)

# 1. Unsupervised baseline model: train only on the first 14 days.
baseline_cutoff = data["timestamp"].min() + pd.Timedelta(days=14)
historical_baseline = train_data[
    train_data["timestamp"] < baseline_cutoff
]

isolation_model = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "isolation_forest",
            IsolationForest(
                n_estimators=300,
                contamination=0.05,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
)
isolation_model.fit(historical_baseline[ISOLATION_FEATURES])

# 2. Supervised binary detector: normal versus anomalous.
binary_model = make_random_forest()
binary_model.fit(train_data[MODEL_FEATURES], train_data["is_anomaly"])

# 3. Supervised multi-class model: which attack does an anomaly resemble?
anomaly_training_data = train_data[train_data["is_anomaly"] == 1]
type_model = make_random_forest()
type_model.fit(
    anomaly_training_data[MODEL_FEATURES],
    anomaly_training_data["anomaly_type"],
)

# Convert Isolation Forest output into an interpretable 0-1 risk component.
baseline_raw_scores = -isolation_model.decision_function(
    historical_baseline[ISOLATION_FEATURES]
)
low_score = np.quantile(baseline_raw_scores, 0.05)
high_score = np.quantile(baseline_raw_scores, 0.99)

all_isolation_raw = -isolation_model.decision_function(
    data[ISOLATION_FEATURES]
)
isolation_risk = np.clip(
    (all_isolation_raw - low_score) / (high_score - low_score),
    0,
    1,
)

positive_class_index = list(binary_model.classes_).index(1)
supervised_probability = binary_model.predict_proba(
    data[MODEL_FEATURES]
)[ :, positive_class_index ]

# Hybrid risk: 45% unknown-behaviour model, 55% labelled attack model.
data["isolation_risk"] = isolation_risk
data["supervised_anomaly_probability"] = supervised_probability
raw_risk_score = (
    100 * (0.45 * isolation_risk + 0.55 * supervised_probability)
)
data["risk_score"] = apply_cold_start_guard(data, raw_risk_score).round(2)
data["risk_tier"] = data["risk_score"].apply(risk_tier)
data["predicted_anomaly"] = (data["risk_score"] >= 50).astype(int)
data["evaluation_partition"] = np.where(
    data["timestamp"] < test_data["timestamp"].min(), "train", "test"
)

predicted_types = predict_attack_types(data, type_model)
data["predicted_attack_type"] = np.where(
    data["predicted_anomaly"] == 1,
    predicted_types,
    "normal",
)
data["explanation"] = data.apply(explain_event, axis=1)

# Hold-out evaluation of the final hybrid anomaly detector.
holdout = data[data["evaluation_partition"] == "test"]
true_values = holdout["is_anomaly"]
predicted_values = holdout["predicted_anomaly"]

precision, recall, f1_score, _ = precision_recall_fscore_support(
    true_values,
    predicted_values,
    average="binary",
    zero_division=0,
)

metrics = {
    "holdout_events": int(len(holdout)),
    "anomaly_events_in_holdout": int(true_values.sum()),
    "precision": round(float(precision), 4),
    "recall": round(float(recall), 4),
    "f1_score": round(float(f1_score), 4),
    "roc_auc": round(
        float(roc_auc_score(true_values, holdout["risk_score"])),
        4,
    ),
    "alert_rate": round(float(holdout["predicted_anomaly"].mean()), 4),
}

MODEL_PATH.parent.mkdir(exist_ok=True)
METRICS_PATH.parent.mkdir(exist_ok=True)

joblib.dump(
    {
        "isolation_model": isolation_model,
        "binary_model": binary_model,
        "type_model": type_model,
        "isolation_features": ISOLATION_FEATURES,
        "model_features": MODEL_FEATURES,
        "isolation_score_low": low_score,
        "isolation_score_high": high_score,
    }

```

```

    },
    MODEL_PATH,
)

data.to_csv(SCORED_PATH, index=False)

with open(METRICS_PATH, "w", encoding="utf-8") as metrics_file:
    json.dump(metrics, metrics_file, indent=2)

print(f"Saved trained models: {MODEL_PATH}")
print(f"Saved scored events: {SCORED_PATH}")
print(f"Saved hold-out metrics: {METRICS_PATH}")
print("\nHybrid detection metrics:")
for name, value in metrics.items():
    print(f"{name}: {value}")

print("\nTop predicted attack categories:")
print(
    data[data["predicted_anomaly"] == 1]["predicted_attack_type"]
    .value_counts()
    .to_string()
)

if __name__ == "__main__":
    main()

```